

UT.6.1 PROGRAMACIÓN EN BASES DE DATOS: VARIABLES, TIPOS DE DATOS Y ESTRUCTURAS DE CONTROL

1. INTRODUCCIÓN	2
2. FUNDAMENTOS PL/SQL	3
3. SINTAXIS	4
3.1 DELIMITADORES	4
3.2 IDENTIFICADORES	5
3.3 COMENTARIOS	6
3.4 LITERALES	6
3.5 OPERADORES Y PRECEDENCIA DE OPERADORES	7
4. TIPOS DE DATOS SIMPLES	9
4.1 SUBTIPOS	10
5. VARIABLES Y CONSTANTES	11
6. EL BLOQUE PL/SQL	13
6.1 USO DE INSTRUCCIONES DML EN BLOQUES PL/SQL	14
6.1.1 Declaración de variables con %TYPE	15
6.1.2 SELECT ... INTO	15
6.1.3 USO DE INSERT, UPDATE y DELETE	16
7. ESTRUCTURAS DE CONTROL	18
7.1 ESTRUCTURAS DE CONTROL CONDICIONAL	19
7.1.1 ESTRUCTURA IF - THEN	19
7.1.2 ESTRUCTURA IF - THEN - ELSE	19
7.1.3 ESTRUCTURA IF - THEN - ELSIF	20
7.1.4 ESTRUCTURA CASE	21
7.2 ESTRUCTURAS DE CONTROL REPETITIVAS	26
7.2.1 BUCLE LOOP	26
7.2.2 BUCLE WHILE ... LOOP	27
7.2.3 BUCLE FOR	28

1.INTRODUCCIÓN

PL(Procedural Language)/SQL es un lenguaje procedimental estructurado en bloques que amplía la funcionalidad de SQL. Con PL/SQL podemos usar sentencias SQL para manipular datos y sentencias de control de flujo para procesar los datos. Por tanto, PL/SQL combina la potencia de SQL para la manipulación de datos, con la potencia de los lenguajes procedimentales para procesar los datos.

Aunque PL/SQL fue creado por Oracle, hoy día todos los gestores de bases de datos utilizan un lenguaje procedimental muy parecido al ideado por Oracle para poder programar las bases de datos.

Como veremos, en PL/SQL podemos definir variables, constantes, funciones, procedimientos, capturar errores en tiempo de ejecución, anidar cualquier número de bloques, etc. como solemos hacer en cualquier otro lenguaje de programación. Además, por medio de PL/SQL programaremos los disparadores de nuestra base de datos, tarea que no podríamos hacer sólo con SQL.

El motor de PL/SQL acepta como entrada bloques PL/SQL o subprogramas, ejecuta sentencias procedimentales y envía sentencias SQL al servidor de bases de datos.

Dependiendo del SGBD podemos encontrar los siguientes tipos de scripts:

- **Anónimos:** Conjunto de instrucciones (en forma de script) que generalmente no suelen ser reutilizables. Se ejecutan directamente, no se almacenan en la BD y un ejemplo típico de uso serían scripts de prueba o tareas puntuales
- **Procedimientos almacenados:** Conjunto de instrucciones que se guardan en el servidor para utilizarlo cuando sea necesario, identificados a través de un nombre. Se suelen usar para realizar un conjunto de acciones.
- **Funciones:** Conjunto de instrucciones que se almacenan en el servidor para realizar alguna tarea. Se suelen usar para hacer cálculos y devuelven un valor.
- **Triggers:** Programa creado para ejecutarse automáticamente cuando ocurra un evento en nuestra base de datos. Veremos más adelante sus usos más habituales.

La unidad lógica en PL es el bloque. Todos los programas están compuestos por bloques que pueden ser anidados o no.

SQL es no-procedural, lo cual significa que es Oracle quien se preocupará de cómo ejecutar de la mejor manera un requerimiento señalado en una sentencia SQL. No es necesaria la conexión entre varias sentencias porque Oracle las ejecuta una cada vez.

PL/SQL permite una completa manipulación de los datos almacenados en una base Oracle, proporciona comandos de control de transacciones y permite utilizar las funciones de SQL, operadores y pseudocolumnas. Además, PL/SQL soporta tipos de datos de SQL, lo que reduce la necesidad de convertir los datos al pasar de una a otra aplicación.

2.FUNDAMENTOS PL/SQL

PL/SQL no es un lenguaje creado para interactuar con el usuario, sino para trabajar con la BD. Prueba de ello es que no dispone de órdenes para introducir datos por teclado, ni para mostrarlos por pantalla.

Oracle incorpora el paquete DBMS_OUTPUT, que incluye el procedimiento PUT_LINE, que permite visualizar textos por pantalla: DBMS_OUTPUT.PUT_LINE ('Expresión').

La estructura de un bloque de PL es la siguiente:

[DECLARE] - Opcional

- Declaración de variables, cursores y excepciones definidas por el usuario.

BEGIN - Obligatorio

- Sentencias SQL.
- Sentencias de control PL/SQL.

[EXCEPTION] - Opcional

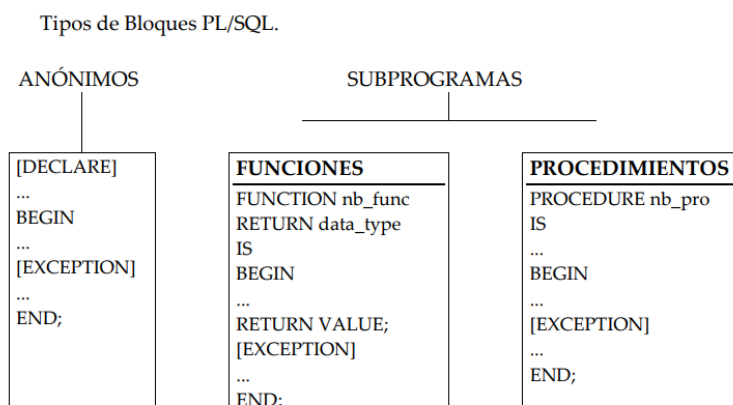
- Acciones a realizar cuando se producen errores.

END; - Obligatorio

/

1. **Sección declarativa (DECLARE):** Contiene todas las variables, constantes y excepciones definidas por el usuario a las que se hace referencia en las secciones ejecutables y excepciones.
2. **Sección ejecutable (BEGIN):** Contiene sentencias SQL para manipular datos de la BD y sentencias PL/SQL para manipular datos del bloque.
3. **Sección para gestión de excepciones (EXCEPTION):** Especifica las acciones que se han de ejecutar cuando aparecen errores y condiciones anormales en la sección ejecutable.

Tipos de bloques PL/SQL:



- **Bloques anónimos:** Son bloques sin nombrar. Son declarados en una aplicación en la que se van a ejecutar y se transfieren al motor PL/SQL para que se ejecuten al momento.
- **Subprogramas:** Son bloques PL/SQL que pueden tomar parámetros y pueden ser invocados. Pueden declararse como procedimientos o como funciones. Generalmente utilizaremos los procedimientos para ejecutar una acción y las funciones para calcular un valor.

Ejemplo de bloque mínimo:

```
SET SERVEROUTPUT ON; --necesario en los clientes SQL * Plus y SQL
Developer para habilitar la salida de dbms_output .

BEGIN
    DBMS_OUTPUT.PUT_LINE('Hola mundo desde PL/SQL');
END;
/
```

PL/SQL no admite bloque vacíos, por lo que la siguiente estructura produce un error:

```
BEGIN
END;
/
```

3. SINTAXIS

PL/SQL es un lenguaje no sensible a las mayúsculas, por lo que será equivalente escribir en mayúsculas o minúsculas, excepto cuando hablemos de literales de tipo cadena o de tipo carácter.

Una sentencia PL/SQL puede comenzar en una línea y acabar en una línea posterior, siendo el delimitador de fin de sentencia el punto y coma (;).

Dentro de una sentencia, los identificadores, literales, comentarios, etc. deben ir separados por uno o más espacios.

3.1 DELIMITADORES

Un **delimitador** es un símbolo simple o compuesto que tiene un significado especial dentro de PL/SQL. Pueden representar operaciones entre tipos de datos, delimitar comentarios, etc. Algunos de los más utilizados son los siguientes:

Delimitadores simples.		Delimitadores compuestos.	
Símbolo.	Significado.	Símbolo.	Significado.
+	Suma.	**	Exponenciación.
%	Indicador de atributo.	<>	Distinto.
.	Selector.	!=	Distinto.
/	División.	<=	Menor o igual.
(	Delimitador de lista.	>=	Mayor o igual.
)	Delimitador de lista.	..	Rango.
:	Variable host.		Concatenación.
,	Separador de elementos.	<<	Delimitador de etiquetas.
*	Producto.	>>	Delimitador de etiquetas.
"	Delimitador de identificador acotado.	--	Comentario de una línea.
=	Igual relacional.	/*	Comentario de varias líneas.
<	Menor.	*/	Comentario de varias líneas.
>	Mayor.	:=	Asignación.
@	Indicador de acceso remoto.	=>	Selector de nombre de parámetro.
;	Terminador de sentencias.		
-	Resta/negación		

Oracle llama *delimitadores* a todos los símbolos especiales del lenguaje PL/SQL. Dentro de ellos hay operadores, separadores, comentarios y otros símbolos. No todos los delimitadores son operadores, pero todos los operadores son delimitadores.

<https://docs.oracle.com/en/database/oracle/oracle-database/26/lnpls/lexical-units.html>

3.2 IDENTIFICADORES

En cuanto a los **identificadores** en PL/SQL, como en cualquier otro lenguaje de programación, son utilizados para nombrar elementos de nuestros programas. A la hora de utilizar los identificadores debemos tener en cuenta los siguientes aspectos:

- Un identificador es una letra seguida opcionalmente de letras, números, \$, _, #.
- No podemos utilizar como identificador una **palabra reservada**.
- Ejemplos válidos: X, A1, codigo_postal.
- Ejemplos no válidos: rock&roll, on/off.
- PL/SQL nos permite además definir los identificadores **acotados**, en los que podemos usar cualquier carácter con una longitud máxima de 30 y deben estar delimitados por ". Ejemplo: "X*Y".
- En PL/SQL existen algunos **identificadores predefinidos** y que tienen un significado especial ya que nos permitirán darle sentido sintáctico a nuestros programas. Estos identificadores son las palabras reservadas y no las podemos utilizar como identificadores en nuestros programas. Ejemplo: IF, THEN, ELSE ...
- Algunas palabras reservadas para PL/SQL no lo son para SQL, por lo que podríamos tener una tabla con una columna llamada 'type' por ejemplo, que nos daría un error de compilación al referirnos a ella en PL/SQL. La solución sería acotarlos. SELECT "TYPE" ...

3.3 COMENTARIOS

En PL/SQL podemos utilizar dos tipos de **comentarios**:

Los comentarios de **una línea** se expresaran por medio del delimitador --. Ejemplo:

```
a:=b;  --asignación
```

Los comentarios de **varias líneas** se acotan por medio de los delimitadores /* y */. Ejemplo:

```
/* Primera línea de comentarios.
   Segunda línea de comentarios. */
```

3.4 LITERALES

Los **literales** se utilizan en las comparaciones de valores o para asignar valores concretos a los identificadores que actúan como variables o constantes. Para expresar estos literales tendremos en cuenta que:

- Los literales numéricos se expresarán por medio de notación decimal o de notación exponencial. Ejemplos: 234, +341, 2e3, -2E-3, 7.45, 8.1e3.
- Los literales tipo carácter y tipo cadena se deben delimitar con unas comillas simples.
- Los literales lógicos son TRUE y FALSE.
- El literal NULL que expresa que una variable no tiene ningún valor asignado.

3.5 OPERADORES Y PRECEDENCIA DE OPERADORES

Como en cualquier otro lenguaje, es aconsejable utilizar los paréntesis para aclarar la precedencia de operadores en expresiones, así como para alterar la precedencia de los mismos, ya que las operaciones encerradas entre paréntesis tienen mayor precedencia. En la tabla siguiente se muestra la precedencia de los operadores de mayor a menor.

Operador.	Operación.
** , NOT	Exponenciación, negación lógica.
+, -	Identidad, negación.
*, /	Multiplicación, división.
+, -,	Suma, resta y concatenación.
=, !=, <, >, <=, >=, IS NULL, LIKE, BETWEEN, IN	Comparaciones.
AND	Conjunción lógica
OR	Disyunción lógica.

Además, en PL/SQL tenemos disponibles todas las funciones predefinidas del sistema, que coinciden en su mayoría con las ya estudiadas en SQL. En las siguientes imágenes puedes repasar algunas de ellas:

<u>Funciones</u>	<u>Descripción</u>
ABS(n)	Devuelve el valor absoluto de un número.
CEIL(n)	Devuelve el entero más pequeño o igual al número.
COS(n)	Coseno de n.
EXP(n)	Número e elevado a n.
FLOOR(n)	Entero más grande o igual a n.
LN(n)	Devuelve el logaritmo neperiano de n.
LOG(n1,n2)	Logaritmo en base n1 de n2.
MOD(n1,n2)	Devuelve el resto de dividir n1 entre n2.
POWER(n1,n2)	Devuelve n1 elevado a n2.
ROUND(n1,n2)	Redondea n1 con n2 decimales.
SVGN(n)	Devuelve -1 si el número es menor de 0, 0 si es igual a 0 y 1 si es
SIN(n)	mayor de 0. Seno de n.
SQRT(n)	Raíz cuadrada de n, que ha de ser siempre mayor de 0.
TAN(n)	Tangente de n.
TRUNC(n1,n2)	Trunca n1 con n2 decimales.

<u>Funciones</u>	<u>Descripción</u>
ASCII(C)	Devuelve el ASCII del carácter.
CHR(n)	Devuelve el carácter correspondiente al número n.
CONCAT (c1, c2)	Concatena 2 cadenas.
INITCAP(c)	Transforma la primera letra a mayúscula.
LENGHT(c)	Longitud de la cadena.
LOWER(c)	Transforma a minúsculas.
LPAD (c1, n, c2)	Justifica a la derecha el valor del carácter c1 n posiciones y los blancos los reemplaza por el carácter c2.
LTRIM(c1, c2)	Suprime caracteres a la izquierda hasta el primer carácter que no está en c2. Si se omite c2 lo pone todo en blanco.
REPLACE(c1,c2,c3)	Devuelve c1 con la cadena c2, reemplazada por c3 y así sucesivamente.
RPAD(c1,n,c2)	Iguala a la derecha.
RTRIM(c1,c2)	Igual que LTRIM pero por la derecha.
SUBSTR(c1,m,n)	Subtrae de la cadena c1, y desde el carácter m, n caracteres.
TRANSLATE(c1,c2,c3)	Devuelve la cadena c1 con cada carácter de c2 que contenga reemplazado por el carácter de c3 con el que se corresponda.
UPPER(c)	Convierte c a mayúsculas.

<u>Funciones</u>	<u>Descripción</u>
TO_CHAR(c1,'fmt')	Convierte un número o fecha a cadena de caracteres.
TO_DATE(c1,'fmt')	Convierte una cadena de caracteres a un número.
TO_NUMBER(c1,'fmt')	Convierte una cadena de caracteres a una fecha.

<u>Funciones</u>	<u>Descripción</u>
ADD_MONTH(fecha,n)	Agrega meses a una fecha.
LAST_DAY(fecha)	Último día del mes.
MONTHS_BETWEEN(fecha1,fecha2)	Número de meses entre dos fechas.
NEXT_DAY(fecha, 'caracter')	Próximo día de la fecha especificada.

<u>Funciones</u>	<u>Descripción</u>
DUMP(expr)	Devuelve la expresión en el formato interno de Oracle.
GREATEST(lista de valores)	Da como resultado el valor más grande de la lista.
LEAST (lista de valores)	Da como resultado el valor más pequeño de la lista.
NVL(expr1, expr2)	Reemplaza los nulos por otro valor.
USER	Devuelve al usuario conectado.
USERENV(opción)	Devuelve información sobre el usuario. Las opciones pueden ser: • SESSIONID: Se identifica la sesión. • TERMINAL: Identificador del terminal. • LANGUAGE: Lenguaje activo.
DECODE	Realiza la función de IF/THEN/ELSE.

4. TIPOS DE DATOS SIMPLES

En cualquier lenguaje de programación, las variables y las constantes tienen un tipo de dato asignado (bien sea explícitamente o implícitamente). Dependiendo del tipo de dato el lenguaje de programación sabrá la estructura que utilizará para su almacenamiento, las restricciones en los valores que puede aceptar, la precisión del mismo, etc.

En PL/SQL contamos con todos los tipos de datos simples utilizados en SQL y algunos más. En este apartado vamos a enumerar los más utilizados.

Numéricos.

NUMBER: Tipo de dato numérico para almacenar números racionales. Podemos especificar su escala (-84 .. 127) y su precisión (1 .. 38). La escala indica cuándo se redondea y hacia dónde. Ejemplos. escala=2: 8.234 -> 8.23, escala=-3: 7689 -> 8000. Si se define un **NUMBER(6, 2)** estamos indicando que son 6 dígitos numéricos de los cuales 2 son decimales. PL/SQL también define algunos **subtipos** como: DEC, DECIMAL, DOUBLE PRECISION, FLOAT, INTEGER, INT, NUMERIC, REAL, SMALLINT.

PLS_INTEGER: Tipo de datos numérico cuyo rango es el mismo que el del tipo de dato **BINARY_INTEGER**, pero que su representación es distinta por lo que las operaciones aritméticas llevadas a cabo con los mismos serán más eficientes que en los dos casos anteriores.

Alfanuméricos.

CHAR(n): Array de n caracteres, máximo 2000 bytes. Si no especificamos longitud sería 1. Ocupa en memoria n caracteres sin importar su contenido, por lo que es raro su uso.

RAW: Array de bytes con un número máximo de 2000. Uso poco frecuente, con datos binarios pequeños.

VARCHAR2(n): Tipo de dato para almacenar cadenas de longitud variable con un máximo de 32760. Ocupa en memoria hasta n caracteres. Grandes objetos.

BFILE: Puntero a un fichero del Sistema Operativo.

BLOB: Objeto binario con una capacidad de 4 GB.

CLOB: Objeto carácter con una capacidad de 2 GB.

Otros.

BOOLEAN: TRUE/FALSE. Recordad que este tipo no existe en SQL, solo en PL/SQL.

DATE: Tipo de dato para almacenar valores fecha/hora desde el 1 enero de 4712 a.c. hasta el 31 diciembre de 4712 d.c.

Hemos visto los tipos de datos simples más usuales. Los tipos de datos compuestos los dejaremos para posteriores apartados.

Atributo %TYPE.

Las variables PL/SQL se suelen declarar para almacenar y manipular los datos que tenemos guardados en nuestra Base de Datos. Cuando declares variables PL/SQL para guardar los valores de columnas, debes asegurarte de que la variable tenga la precisión (tamaño) y el tipo de dato correcto. Si no es así se puede producir un error durante la ejecución.

Cuando se utiliza el atributo %TYPE para declarar una variable estamos indicando que tiene el mismo tipo de datos que la variable especificada. Normalmente se relaciona con una columna de la BD indicando el nombre de la tabla de base de datos y el nombre de la columna. Si se hace referencia a una variable declarada anteriormente, se indicará el nombre de la variable previamente declarada a la variable por declarar.

Una ventaja del uso de este atributo, es que si cambia la definición de una columna de la BD, no será necesario cambiar la declaración de la variable.

Ejemplo de declaración de variables:

```
DECLARE
    nombre_departamento departamento.nomdep%type; -- la variable
nombre_departamento tomará el mismo tipo que la columna nomdep de la
tabla departamento
    vx number(5,2); -- Variable numérica de 5 dígitos, dos de los cuales
son decimales
    vy vx%type; -- la variable vy tomará el mismo tipo de dato que tenga
la variable vx
BEGIN
    NULL;
END;
/
```

4.1 SUBTIPOS

Un subtipo en PL/SQL permite cambiar el nombre de un tipo para aumentar la legibilidad de nuestros programas. Los tipos de operaciones aplicables a estos subtipos serán las mismas que los tipos de datos de los que proceden. La sintaxis será:

```
SUBTYPE nombre_subtipo IS tipo_base;
```

donde tipo_base será cualquier tipo de dato en PL/SQL. A la hora de especificar el tipo base, podemos utilizar el modificador %TYPE para indicar el tipo de dato de una variable.

```

DECLARE
    SUBTYPE coddep IS departamento.coddep%TYPE;
    variable coddep;
BEGIN
    ...
END;
/

```

Los subtipos son intercambiables con su tipo base. También son intercambiables si tienen el mismo tipo base o si su tipo base pertenece a la misma familia:

```

DECLARE
    SUBTYPE numero IS NUMBER;
    numero_tres_digitos NUMBER(3);
    mi_numero_de_la_suerte numero;
    SUBTYPE encontrado IS BOOLEAN;
    SUBTYPE resultado IS BOOLEAN;
    lo_he_encontrado encontrado;
    resultado_busqueda resultado;
    SUBTYPE literal IS CHAR;
    SUBTYPE sentencia IS VARCHAR2;
    literal_nulo literal;
    sentencia_vacia sentencia;
BEGIN
    ...
    numero_tres_digitos := mi_numero_de_la_suerte; --legal
    ...
    lo_he_encontrado := resultado_busqueda; --legal
    ...
    sentencia_vacia := literal_nulo; --legal
    ...
END;

```

5. VARIABLES Y CONSTANTES

Una **variable** en PL/SQL es una **ubicación de almacenamiento con nombre** que **contiene un valor** de un **tipo de dato** específico (como NUMBER, VARCHAR2, DATE), cuyo valor puede cambiar durante la ejecución. Se declaran obligatoriamente en la sección DECLARE de un bloque antes de usarse, permitiendo almacenar y manipular datos.

Por su parte, una **constante** en PL/SQL es un **elemento de datos con nombre cuyo valor se define al declararse y permanece inalterable durante toda la ejecución** de un bloque, procedimiento o función.

Para declarar variables o constantes pondremos el **nombre de la variable**, seguido del **tipo** de datos y **opcionalmente una inicialización** con el operador de asignación `:=`. Si es una constante antepone la palabra **CONSTANT** al tipo de dato (lo que querrá decir que no podemos cambiar su valor).

```
nombre_variable [CONSTANT] tipo_de_dato [NOT NULL] [:=expr | DEFAULT  
expr];
```

Podremos sustituir el operador de asignación en las declaraciones por la palabra reservada `DEFAULT`. También podremos forzar a que no sea nula utilizando la palabra `NOT NULL` después del tipo y antes de la asignación. Si restringimos una variable con `NOT NULL` deberemos asignarle un valor al declararla, de lo contrario PL/SQL lanzará la excepción `VALUE_ERROR` (más adelante veremos lo que son las excepciones, pero como adelanto te diré que es un error en tiempo de ejecución).

```
DECLARE
    hoy DATE := sysdate;
    pi CONSTANT NUMBER:= 3.1415;
    id SMALLINT NOT NULL; --ilegal, no está inicializada
    id SMALLINT NOT NULL := 9999; --legal
    nombre VARCHAR2(30) := 'Alejandro Magno';
    num NUMBER(3) DEFAULT 4; -- también se puede utilizar DEFAULT para
inicializar una variable
BEGIN
    NULL;
END;
/
```

El alcance(o ámbito) y la visibilidad de las variables en PL/SQL será el típico de los lenguajes estructurados basados en bloques, aunque eso lo veremos más adelante.

Conversión de tipos.

Aunque en PL/SQL existe una conversión implícita de tipos para tipos parecidos, siempre es aconsejable utilizar la conversión explícita de tipos por medio de funciones de conversión (`TO_CHAR`, `TO_DATE`, `TO_NUMBER`, ...) y así evitar resultados inesperados.

```
DECLARE
    v_num NUMBER := TO_NUMBER('123');
    v_fecha DATE := TO_DATE('2024-01-01','YYYY-MM-DD');
    v_texto VARCHAR2(10) := TO_CHAR(SYSDATE, 'DD/MM/YYYY');
```

6. EL BLOQUE PL/SQL

Ya hemos visto las unidades léxicas que componen PL/SQL, los tipos de datos que podemos utilizar y cómo se definen las variables y las constantes. Ahora vamos a ver la **unidad básica en PL/SQL** que es el **bloque**.

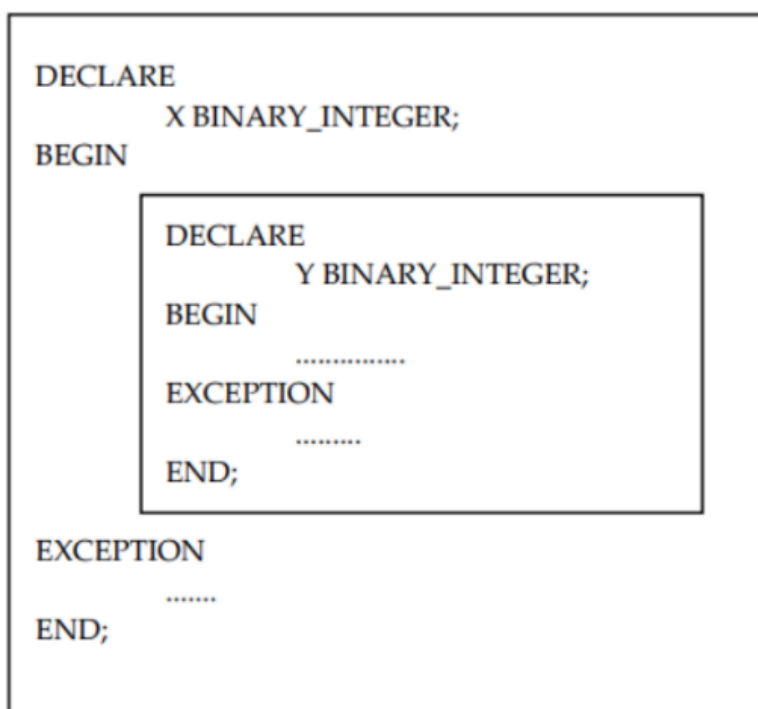
Como ya hemos comentado, un bloque PL/SQL consta de **tres zonas**:

- **Declaraciones:** definiciones de variables, constantes, cursores y excepciones.
- **Proceso:** zona donde se realizará el proceso en sí, conteniendo las sentencias ejecutables.
- **Excepciones:** zona de manejo de errores en tiempo de ejecución.

La sintaxis es la siguiente:

```
[DECLARE]
    Declaración de variables, constantes, cursores y excepciones
BEGIN
    Sentencias ejecutables
[EXCEPTION]
    Manejadores de excepciones
END;
/
```

Los bloques PL/SQL pueden **anidarse** a cualquier nivel.



Como hemos comentado anteriormente, el **ámbito y la visibilidad** de las variables es la normal en un **lenguaje procedimental**.

Las referencias a un identificador son resueltas de acuerdo a su alcance y visibilidad dentro de un programa. El alcance de un identificador es aquella región de la unidad de programa (bloque, subprograma o paquete) desde la cual se puede referenciar al identificador.

Un identificador es visible sólo en las regiones en que se puede referenciar.

Los identificadores declarados en un bloque de PL/SQL se consideran locales al bloque y globales a todos sus sub-bloques o bloques anidados. De esto se desprende que un mismo identificador no se puede declarar dos veces en un mismo bloque pero sí en varios bloques diferentes, cuántas veces se desee.

Por ejemplo, en el siguiente fragmento de código se declara la variable aux en ambos bloques, pero en el bloque anidado aux con valor igual a 10 actúa de variable **global** y aux con valor igual a 5 actúa como variable **local**, por lo que en la comparación evaluaría a FALSE, ya que al tener el mismo nombre la visibilidad dominante sería la de la variable local.

```
DECLARE
    aux number := 10; -- Variable global
BEGIN
    DECLARE
        aux number := 5; -- Variable local al bloque donde es definida
    BEGIN
        ...
        IF aux = 10 THEN --evalúa a FALSE, no entraría
        ...
    END;
END;
```

6.1 USO DE INSTRUCCIONES DML EN BLOQUES PL/SQL

Una de las principales ventajas de PL/SQL es que permite combinar **lógica procedimental** (variables, condicionales, bucles) con **sentencias SQL** (SELECT, INSERT, UPDATE, DELETE).

Esto permite crear programas que lean datos de la base de datos, los procesen y los modifiquen. Hasta ahora hemos trabajado con SQL de forma interactiva, escribíamos la sentencia y obteníamos los resultados en pantalla. Para trabajar con SQL dentro de un programa trabajaremos con SQL embebido o incrustado. Los datos devueltos de la consulta los guardaremos en variables y estructuras definidas en el lenguaje anfitrión, en nuestro caso PLSQL, para utilizarlas como convenga. Para ello añadiremos alguna cláusula a la sentencia SELECT.

PL/SQL soporta el Lenguaje de Manipulación de Datos (DML) y los comandos de control de transacciones. Se pueden utilizar los comandos DML para modificar los datos de una tabla de Base de Datos.

6.1.1 Declaración de variables con %TYPE

Cuando trabajamos con datos almacenados en tablas, es recomendable declarar las variables utilizando el atributo %TYPE, ya que:

- garantiza que la variable tiene el mismo tipo que la columna,
- evita errores si la definición de la tabla cambia,
- mejora la legibilidad del código.

Ejemplo sobre la base de datos EMPRESA:

```
v_nombre centro.NomCen%TYPE;  
v_codigo departamento.CodDep%TYPE;
```

6.1.2 SELECT ... INTO

Para recuperar datos dentro de un bloque PL/SQL utilizamos la sintaxis:

```
SELECT lista_campos INTO {nombre_variable[, nombre_variable]...|  
nombre_registro}  
FROM tabla  
[WHERE condición];
```

donde

- lista_campos, contendrá al menos una columna, y puede incluir expresiones SQL, funciones de fila, o funciones de grupo.
- La cláusula INTO es obligatoria y se especifica entre SELECT y cláusula FROM.
- nombre_variable, variable donde se guardará el valor recuperado. Se deben especificar tantas variables como campos se indiquen en la lista de campos, debe haber correspondencia en posición y en tipo de datos.
- nombre_registro, un dato compuesto de PL/SQL donde se guardarán los valores recuperados. Veremos tipos de datos compuestos más adelante.

Las instrucciones SELECT dentro de un programa PL/SQL deben devolver una sola fila. Una consulta que devuelve más de una fila o ninguna fila genera un error de tipo, respectivamente TOO_MANY_ROWS o NO_DATA_FOUND. Si queremos recuperar más de una fila necesitaremos una estructura de datos llamada cursor que se verá más adelante.

Las instrucciones DML, INSERT, UPDATE y DELETE, se ejecutan utilizando la misma sintaxis que en SQL.

6.1.3 USO DE INSERT, UPDATE y DELETE

Las sentencias DML se utilizan igual que en SQL, pero pueden combinar con variables PL/SQL:

```
INSERT INTO tabla VALUES (...);  
UPDATE tabla SET columna = valor WHERE ...;  
DELETE FROM tabla WHERE ...;
```

Tras ejecutar una sentencia DML, PL/SQL actualiza automáticamente el atributo SQL%ROWCOUNT, que indica cuántas filas se han modificado.

Ejemplos:

- Insertar un departamento usando variables %TYPE:

```
SET SERVEROUTPUT ON;  
  
DECLARE  
    v_codcen    centro.CodCen%TYPE := 'OFZL';  
    v_nomcen    centro.NomCen%TYPE;  
    v_coddep    departamento.CodDep%TYPE := 'DESZL';  
    v_nomdep    departamento.NomDep%TYPE := 'Desarrollo de ventas';  
BEGIN  
    -- Recuperamos el nombre del centro  
    SELECT NomCen  
    INTO v_nomcen  
    FROM centro  
    WHERE CodCen = v_codcen;  
  
    DBMS_OUTPUT.PUT_LINE('Insertando departamento en el centro: ' ||  
v_nomcen);  
  
    INSERT INTO departamento (CodDep, CodCen, NomDep, PreAnu, TiDir)  
    VALUES (v_coddep, v_codcen, v_nomdep, 100000, 'F');  
  
    DBMS_OUTPUT.PUT_LINE('Filas insertadas: ' || SQL%ROWCOUNT);  
END;  
/
```


- Actualizar el salario de un empleado según su número de hijos:

```
SET SERVEROUTPUT ON;

DECLARE
    v_codemp    empleado.CodEmp%TYPE := 29;
    v_salario    empleado.SalEmp%TYPE;
    v_hijos      NUMBER;
BEGIN
    -- Obtener salario actual
    SELECT SalEmp
    INTO v_salario
    FROM empleado
    WHERE CodEmp = v_codemp;

    -- Contar hijos
    SELECT COUNT(*)
    INTO v_hijos
    FROM hijo
    WHERE CodEmp = v_codemp;

    -- Aumentar salario según número de hijos
    UPDATE empleado
    SET SalEmp = SalEmp + (v_hijos * 50)
    WHERE CodEmp = v_codemp;

    DBMS_OUTPUT.PUT_LINE('Empleado ' || v_codemp || ' tenía salario ' ||
v_salario);
    DBMS_OUTPUT.PUT_LINE('Número de hijos: ' || v_hijos);
    DBMS_OUTPUT.PUT_LINE('Filas actualizadas: ' || SQL%ROWCOUNT);
END;
/
```

Cuando ejecutamos sentencias DML (INSERT, UPDATE, DELETE) dentro de un bloque PL/SQL, los cambios **no se confirman automáticamente**. En Oracle, toda modificación forma parte de una **transacción**, que permanece abierta hasta que se ejecute explícitamente:

- COMMIT ; confirma los cambios de forma permanente.
- ROLLBACK ; deshace todos los cambios realizados desde el último COMMIT.

Esto significa que:

Aunque un bloque PL/SQL haya terminado correctamente,

- **los cambios no son definitivos hasta que se haga COMMIT.**
- Si ocurre un error y no se controla, Oracle puede deshacer la transacción automáticamente.
- Si el programador quiere garantizar consistencia, debe usar COMMIT o ROLLBACK dentro del bloque.

Tanto el COMMIT como el ROLLBACK pueden ejecutarse dentro del bloque PL/SQL o posteriormente desde la sesión SQL.

Ejemplo:

```
SET SERVEROUTPUT ON;

DECLARE
    v_codemp empleado.CodEmp%TYPE := 29;
    v_salario empleado.SalEmp%TYPE;
BEGIN
    SELECT SalEmp
    INTO v_salario
    FROM empleado
    WHERE CodEmp = v_codemp;

    UPDATE empleado
    SET SalEmp = SalEmp + 200
    WHERE CodEmp = v_codemp;

    DBMS_OUTPUT.PUT_LINE('Salario actualizado. Pendiente de
confirmación.');
```

COMMIT; -- Sin esto, el cambio no sería definitivo

```
    DBMS_OUTPUT.PUT_LINE('Nuevo salario confirmado.');
```

END;

/

7. ESTRUCTURAS DE CONTROL

Como cualquier lenguaje procedimental, PL/SQL dispone de una serie de instrucciones que nos permiten modificar el flujo normal de ejecución del bloque (que es de arriba hacia abajo). Estas instrucciones permiten tomar decisiones que hacen que llevemos a cabo unas acciones u otras dependiendo de unas circunstancias, o repetir una serie de acciones un número dado de veces o hasta que se cumpla una condición. PL/SQL dispone de dos tipos de estructuras de control: condicionales e iterativas.

7.1 ESTRUCTURAS DE CONTROL CONDICIONAL

A menudo es necesario tomar alternativas de acción dependiendo de las circunstancias. Las estructuras de control condicional nos permiten llevar a cabo una acción u otra dependiendo de una condición.

7.1.1 ESTRUCTURA IF - THEN

Forma más simple de las sentencias de control condicional. Si la evaluación de la condición es TRUE, entonces se ejecuta la secuencia de sentencias encerradas entre el THEN y el final de la sentencia (END IF). Sintaxis:

```
IF condicion THEN
    secuencia_de_sentencias;
END IF;
```

La secuencia de sentencias es ejecutada sólo si la condición es verdadera. Si la condición es falsa o nula no realiza nada.

```
SET SERVEROUTPUT ON
DECLARE
    a INTEGER := 10;
    b INTEGER := 7;
BEGIN
    IF a > b THEN
        dbms_output.put_line(a || ' es mayor'); -- Como la función
put_line solo imprime un valor utilizamos el operador de concatenación
    END IF;
END;
/
```

7.1.2 ESTRUCTURA IF - THEN - ELSE

Con esta forma de la sentencia ejecutaremos la primera secuencia de sentencias si la condición se evalúa a TRUE, y en caso contrario ejecutaremos la segunda secuencia de sentencias (posterior al ELSE). Sintaxis:

```
IF condicion THEN
    Secuencia_de_sentencias1;
ELSE
    Secuencia_de_sentencias2;
END IF;
```

Ejemplos:

```

SET SERVEROUTPUT ON
DECLARE
    a INTEGER := 10;
    b INTEGER := 7;
BEGIN
    IF a > b THEN
        dbms_output.put_line(a || ' es mayor');
    ELSE
        dbms_output.put_line(b || ' es mayor o iguales');
    END IF;
END;
/

```

Podemos anidar estructuras IF - THEN - ELSE como se ve en el ejemplo:

```

IF tipo_trans = 'CR' THEN
    UPDATE cuentas SET balance = balance + credito WHERE ...
ELSE
    IF nuevo_balance >= minimo_balance THEN
        UPDATE cuentas SET balance = balance - debito
        WHERE ...
    ELSE
        UPDATE cuentas SET balance = balance - minimo_balance
        WHERE ...
    END IF;
END IF;

```

7.1.3 ESTRUCTURA IF - THEN - ELSIF

Con esta última forma de la sentencia condicional podemos hacer una selección múltiple. Si la evaluación de la condición 1 da TRUE, ejecutamos la secuencia de sentencias 1, sino evaluamos la condición 2. Si ésta evalúa a TRUE ejecutamos la secuencia de sentencias 2 y así para todos los ELSIF que haya. El último ELSE es opcional, y ejecuta su secuencia de sentencias si no se cumple ninguna de las condiciones anteriores. Sintaxis:

```

IF condicion1 THEN
    Secuencia_de_sentencias1;
ELSIF condicion2 THEN
    Secuencia_de_sentencias2;
...
[ELSE]
    Secuencia_de_sentencias;
END IF;

```

Si la primera condición es falsa o nula, la cláusula ELSIF verifica una nueva condición. Cada sentencia IF puede poseer un número indeterminado de cláusulas ELSIF; la palabra clave ELSE que se encuentra al final es opcional.

Ejemplo:

```
DECLARE
    operacion VARCHAR2(8) := 'PRODUCTO';
    arg1 NUMBER := 2;
    arg2 NUMBER := 4;
    resultado NUMBER;
BEGIN
    IF (operacion = 'SUMA') THEN
        resultado := arg1 + arg2;
    ELSIF (operacion = 'RESTA') THEN
        resultado := arg1 - arg2;
    ELSIF (operacion = 'PRODUCTO') THEN
        resultado := arg1 * arg2;
    ELSIF (arg2 <> 0) AND (operacion = 'DIVISION') THEN
        resultado := arg1 / arg2;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Operación no permitida');
    END IF;
    DBMS_OUTPUT.PUT_LINE('Resultado: ' || resultado);
END;
/
```

7.1.4 ESTRUCTURA CASE

Se utiliza de forma similar al “Switch case de Java”, que al final es otra forma de expresar una estructura IF – THEN – ELSIF. Permite representar n sentencias IF anidadas, y es más fácil de interpretar cuando se compara con varios valores, permitiendo sustituir a las sentencias IF encadenadas. Se puede utilizar de dos formas:

- Utilizando un selector y un manejador WHEN para cada posible valor de ese selector:

```
CASE selector
    WHEN expresion_1 THEN
        ordenes;
    [WHEN expresion_2 THEN
        ordenes;
    ...
    [WHEN expresion_N THEN
```

```

        ordenes;]
    [ ELSE
        ordenes;]
END CASE ;

```

- Utilizando condiciones de búsqueda:

```

CASE
    WHEN condición_1 THEN
        ordenes;
    [WHEN condición_2 THEN
        ordenes;
    ...
    WHEN condición_N THEN
        ordenes;]
    [ ELSE
        ordenes;]
END CASE ;

```

Ejemplos

En cualquiera de las dos formas, antes de terminar la sentencia CASE se puede especificar ELSE por si no se ha cumplido ninguna de las condiciones o el valor del selector no ha coincidido con ninguno de los evaluados. Las condiciones, condición_1 a condición_N, son analizadas en el mismo orden en que aparecen listadas y, en el momento en que una de estas condiciones se cumple como verdadera, la sentencia CASE devuelve el resultado correspondiente y deja de analizar el resto de condiciones. En este punto no es necesario para la ejecución como ocurre en Java (break;)

Ejemplo:

```

DECLARE
nota INTEGER := 8; -- Se podría especificar nota INTEGER := &nota
BEGIN
    CASE
        WHEN nota IN (1,2,3,4) THEN
            DBMS_OUTPUT.PUT_LINE('No superado');
        WHEN nota = 5 THEN
            DBMS_OUTPUT.PUT_LINE('Suficiente');
        WHEN nota = 6 THEN
            DBMS_OUTPUT.PUT_LINE('Bien');
        WHEN nota IN (7,8) THEN
            DBMS_OUTPUT.PUT_LINE('Notable');
        WHEN nota IN (9,10) THEN

```

```

        DBMS_OUTPUT.PUT_LINE('Sobresaliente');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Error, no es una nota');
    END CASE;
END;
/

```

Para pedir datos por teclado en PL/SQL se utilizarán variables de sustitución escribiendo el carácter especial '&' y a continuación un identificador. Si el dato que se va a recibir es una cadena formada por caracteres alfanuméricos, se deben incluir las comillas al definir la expresión.

```

DECLARE
    cadena varchar2(25) := '&cad';
BEGIN
    DBMS_OUTPUT.PUT_LINE(cadena);
END;
/

```

Ejercicios sobre BBDD HR:

- Realizar un bloque anónimo de PL/SQL que solicite por teclado dos identificadores de empleados y muestre por pantalla si ambos empleados trabajan en el mismo país o no.
- Realizar un bloque anónimo de PL/SQL que solicite por teclado el identificador de un empleado y muestre por pantalla un mensaje indicando:
 - “Salario bajo” si el salario es menor de 5000
 - “Salario medio” si está entre 5000 y 10000
 - “Salario alto” si es mayor de 10000
- Utilizando una estructura CASE, realizar un bloque anónimo de PL/SQL que solicite por teclado el identificador de un empleado y muestre por pantalla el tipo de contrato según su JOB_ID:
 - IT_PROG → “Informático”
 - SA_REP → “Comercial”
 - ST_CLERK → “Administrativo”
 - Cualquier otro → “Otro puesto”

Soluciones:

```

SET SERVEROUTPUT ON;

DECLARE
    v_emp1    employees.employee_id%TYPE := &emp1;
    v_emp2    employees.employee_id%TYPE := &emp2;

    v_pais1    locations.country_id%TYPE;
    v_pais2    locations.country_id%TYPE;
BEGIN
    -- País del primer empleado
    SELECT l.country_id
    INTO v_pais1
    FROM employees e
    JOIN departments d ON e.department_id = d.department_id
    JOIN locations l   ON d.location_id = l.location_id
    WHERE e.employee_id = v_emp1;

    -- País del segundo empleado
    SELECT l.country_id
    INTO v_pais2
    FROM employees e
    JOIN departments d ON e.department_id = d.department_id
    JOIN locations l   ON d.location_id = l.location_id
    WHERE e.employee_id = v_emp2;

    -- Comparación
    IF v_pais1 = v_pais2 THEN
        DBMS_OUTPUT.PUT_LINE('Ambos empleados trabajan en el mismo país: '
|| v_pais1);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Los empleados trabajan en países
distintos.');
```

```

        DBMS_OUTPUT.PUT_LINE('Empleado ' || v_emp1 || ': ' || v_pais1);
        DBMS_OUTPUT.PUT_LINE('Empleado ' || v_emp2 || ': ' || v_pais2);
    END IF;
END;
/

```



```

SET SERVEROUTPUT ON;

DECLARE
    v_emp_id    employees.employee_id%TYPE := &emp;
    v_salario   employees.salary%TYPE;
BEGIN
    SELECT salary
    INTO v_salario
    FROM employees
    WHERE employee_id = v_emp_id;

    IF v_salario < 5000 THEN
        DBMS_OUTPUT.PUT_LINE('Salario bajo');
    ELSIF v_salario BETWEEN 5000 AND 10000 THEN
        DBMS_OUTPUT.PUT_LINE('Salario medio');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Salario alto');
    END IF;
END;
/

```

```

SET SERVEROUTPUT ON;

DECLARE
    v_emp_id    employees.employee_id%TYPE := &emp;
    v_job        employees.job_id%TYPE;
BEGIN
    SELECT job_id
    INTO v_job
    FROM employees
    WHERE employee_id = v_emp_id;

    CASE v_job
        WHEN 'IT_PROG' THEN
            DBMS_OUTPUT.PUT_LINE('Informático');
        WHEN 'SA_REP' THEN
            DBMS_OUTPUT.PUT_LINE('Comercial');
        WHEN 'ST_CLERK' THEN
            DBMS_OUTPUT.PUT_LINE('Administrativo');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Otro puesto');
    END CASE;
END;
/

```

7.2 ESTRUCTURAS DE CONTROL REPETITIVAS

Ya que hemos visto las estructuras de control condicional, veamos ahora las estructuras de control **repetitivo o iterativo**.

7.2.1 BUCLE LOOP

Estas estructuras nos permiten ejecutar una secuencia de sentencias un determinado número de veces. Por defecto, en su forma más sencilla, se corresponde con un bucle infinito:

```
LOOP
    secuencia_de_sentencias;
END LOOP;
```

Con cada iteración del ciclo las sentencias son ejecutadas. Para terminar estos ciclos de ejecución se utiliza la palabra clave **EXIT**. Es posible ubicar varios EXIT dentro del loop:

```
LOOP
    ...
    IF condición THEN
        EXIT;
    END IF;
END LOOP;
```

EXIT hace finalizar el bucle incondicionalmente, por lo que suele incluirse dentro de una estructura condicional adicional. Existe la posibilidad de utilizar la estructura **EXIT WHEN**, que permite terminar el loop de manera condicional. Cuando se encuentra el EXIT, la condición de la cláusula WHEN es evaluada. Si la condición es verdadera el loop es terminado y el control es pasado a la próxima sentencia:

```
LOOP
    ...
    EXIT WHEN condición;
END LOOP;
```

Por tanto, la sentencia EXIT – WHEN reemplaza la utilización de un IF, pero se considera que da lugar a código más legible por lo que recomendamos su uso.

Debemos tener en cuenta que en el interior del bucle debe de haber alguna instrucción que modifique la evaluación de la condición, de forma que alguna vez devuelva TRUE. En caso contrario tendremos un bucle infinito.

Ejemplos:

```
DECLARE
    a integer := 1;
BEGIN
    LOOP
        dbms_output.put_line(a);
        IF a>9 THEN
            EXIT;
        END IF;
        a := a + 1;
    END LOOP;
END;
/
```

```
DECLARE
    a integer := 1;
BEGIN
    LOOP
        dbms_output.put_line(a);
        EXIT WHEN a > 9;
        a := a + 1;
    END LOOP;
END;
/
```

7.2.2 BUCLE WHILE ... LOOP

Este tipo de bucle ejecuta una secuencia de sentencias mientras que una condición sea cierta. Sintaxis:

```
WHILE condición LOOP
    Secuencia_de_sentencias;
END LOOP;
```

La condición se evalúa antes de cada iteración. Si es verdadera se ejecuta la secuencia de órdenes y, si es falsa, el bucle termina. Si la condición del bucle no toma un valor 'true', el bucle no llega a ejecutarse.

De nuevo tenemos que ejecutar alguna instrucción que haga que la condición evalúe a TRUE o tendremos un bucle sin fin.

Ejemplo:

```

DECLARE
    a integer := 1;
BEGIN
    WHILE a < 10 LOOP
        dbms_output.put_line(a);
        a := a+1;
    END LOOP;
END;
/

```

7.2.3 BUCLE FOR

Ejecuta las instrucciones del bucle un número predeterminado de veces según la condición incluida en la declaración. La iteración se produce mientras el contador se encuentra dentro del rango límite inferior - límite superior. Sintaxis:

```

FOR contador IN [REVERSE] limite_inferior..limite_superior LOOP
    Secuencia_de_sentencias;
END LOOP;

```

contador es el índice del bucle, y por defecto es `binary_integer`. No es necesario declararlo.

Los límites del bucle sólo se evalúan una vez, y determinan el número total de iteraciones, en las que el índice varía, entre el límite superior e interior, incrementándose en una unidad. [REVERSE] sirve para inicializar al límite superior e ir decrementando en una unidad. El índice del bucle FOR se incrementa o decrementa automáticamente en una unidad. No es posible especificar un paso distinto de 1 o -1.

```

BEGIN
    FOR a IN 1..10 LOOP -- ascendente de uno en uno
        dbms_output.put_line(a);
    END LOOP;
    FOR a IN REVERSE 1..10 LOOP -- descendente de uno en uno
        dbms_output.put_line(a);
    END LOOP;
END;
/

```

Ejercicios:

- Realizar un bloque anónimo de PL/SQL que inserte **5 empleados nuevos** en la tabla EMPLOYEES.

Los empleados tendrán:

- EMPLOYEE_ID del 300 al 304
 - FIRST_NAME = 'Empleado' || número
 - LAST_NAME = 'Demo'
 - EMAIL = 'EMP' || número
 - HIRE_DATE = SYSDATE
 - JOB_ID = 'IT_PROG'
 - SALARY = 3000 + (número * 100)
-
- Utilizar un **bucle FOR** para generar e insertar los valores.

Realizar un bloque anónimo de PL/SQL que inserte **10 departamentos** en la tabla DEPARTMENTS.

Los departamentos tendrán:

- DEPARTMENT_ID del 300 al 309
- DEPARTMENT_NAME = 'Departamento ' || número
- LOCATION_ID = 1700

Utilizar un **bucle WHILE**.

- Realizar un bloque anónimo de PL/SQL que inserte **puestos de trabajo ficticios** en la tabla JOBS.

Los puestos tendrán:

- JOB_ID = 'TEST_JOB_' || número
- JOB_TITLE = 'Puesto de prueba ' || número
- MIN_SALARY = 1000
- MAX_SALARY = 2000

Insertar puestos desde el número 1 hasta el número 5 utilizando un **bucle LOOP** y finalizar con **EXIT WHEN**.

Soluciones:

```
SET SERVEROUTPUT ON;

BEGIN
  FOR i IN 300..304 LOOP
    INSERT INTO employees (employee_id, first_name, last_name, email,
                          hire_date, job_id, salary)
    VALUES (i,
```

```

        'Empleado' || (i - 299),
        'Demo',
        'EMP' || (i - 299),
        SYSDATE,
        'IT_PROG',
        3000 + ((i - 299) * 100));

        DBMS_OUTPUT.PUT_LINE('Insertado empleado ' || i);
    END LOOP;

    COMMIT;
END;
/

```

```

SET SERVEROUTPUT ON;

DECLARE
    v_dep NUMBER := 300;
BEGIN
    WHILE v_dep <= 309 LOOP
        INSERT INTO departments (department_id, department_name,
location_id)
        VALUES (v_dep,
                'Departamento ' || (v_dep - 299),
                1700);

        DBMS_OUTPUT.PUT_LINE('Insertado departamento ' || v_dep);

        v_dep := v_dep + 1;
    END LOOP;

    COMMIT;
END;
/

```

```

SET SERVEROUTPUT ON;

DECLARE
    v_num NUMBER := 1;
BEGIN
    LOOP

```

```
INSERT INTO jobs (job_id, job_title, min_salary, max_salary)
VALUES ('TEST_JOB_' || v_num,
        'Puesto de prueba ' || v_num,
        1000,
        2000);

DBMS_OUTPUT.PUT_LINE('Insertado TEST_JOB_' || v_num);

EXIT WHEN v_num = 5;
v_num := v_num + 1;
END LOOP;

COMMIT;
END;
/
```